Student Name: Husanpreet Kaur **UID: 24BCY70138**
Branch: AIT-CSE (Cyber Security) **Section/Group: 24AIT_KRG2**
Semester: 4 **Date of Performance: 27/02/2026**
Subject Name: Database Management System **Subject Code: 24CSH-298**

WORKSHEET 7

AIM: To design and implement a materialized view and to compare and analyze execution time and performance differences between simple views, complex views, and materialized views.

S/W Requirement:
- Database Management System: PostgreSQL / Oracle Database Express Edition
- Database Administration Tool: pgAdmin

OBJECTIVES:

- Create simple views
- Create complex views
- Create materialized views
- Compare execution time and performance of each

PROBLEM STATEMENT:

In large-scale enterprise systems, frequent execution of complex queries can significantly affect performance. Optimized mechanisms are required to improve query response time without compromising data consistency.

TABLE STRUCTURE:

[SQL Worksheet]*

```sql
CREATE TABLE emp_table (
    emp_id NUMBER PRIMARY KEY,
    emp_name VARCHAR2(50),
    dept_id NUMBER,
    salary NUMBER
);
```

```
SQL> CREATE TABLE emp_table (
         emp_id NUMBER PRIMARY KEY,
         emp_name VARCHAR2(50),
         dept_id NUMBER,...
Show more...


Table EMP_TABLE created.

Elapsed: 00:00:00.014
```

```sql
CREATE TABLE department (
    dept_id NUMBER PRIMARY KEY,
    dept_name VARCHAR2(50)
);
```

```
SQL> CREATE TABLE department (
         dept_id NUMBER PRIMARY KEY,
         dept_name VARCHAR2(50)
     )



Table DEPARTMENT created.

Elapsed: 00:00:00.015
```

```sql
INSERT INTO department VALUES (1, 'HR');
INSERT INTO department VALUES (2, 'IT');

INSERT INTO emp_table VALUES (101, 'John', 1, 3000);
INSERT INTO emp_table VALUES (102, 'Alice', 2, 5000);
INSERT INTO emp_table VALUES (103, 'Bob', 2, 4000);
INSERT INTO emp_table VALUES (104, 'Emma', 1, 3500);

COMMIT;
```

```
1 row inserted.

Elapsed: 00:00:00.001
```

```
SQL> COMMIT


Commit complete.

Elapsed: 00:00:00.001
```

1. PROBLEM STATEMENT – Simple View

Create a simple view based on a single table.

PROGRAM:

```
CREATE VIEW emp_view AS
SELECT emp_id, emp_name, salary
FROM emp_table;
```

OUTPUT:

```
SQL> CREATE VIEW emp_view AS
     SELECT emp_id, emp_name, salary
     FROM emp_table



View EMP_VIEW created.

Elapsed: 00:00:00.013
```

2. PROBLEM STATEMENT –Complex View

Create a complex view using join and aggregation.

PROGRAM:

```
CREATE VIEW emp_dept_view AS
SELECT d.dept_name, AVG(e.salary) AS avg_salary
FROM emp_table e
JOIN department d ON e.dept_id = d.dept_id
GROUP BY d.dept_name;
```

OUTPUT:

```
SQL> CREATE VIEW emp_dept_view AS
     SELECT d.dept_name, AVG(e.salary) AS avg_salary
     FROM emp_table e
     JOIN department d ON e.dept_id = d.dept_id...
Show more...



View EMP_DEPT_VIEW created.

Elapsed: 00:00:00.009
```

3. PROBLEM STATEMENT – Materialized View

Create a materialized view to store precomputed results.

```
CREATE MATERIALIZED VIEW emp_mv
BUILD IMMEDIATE
REFRESH COMPLETE
AS
SELECT d.dept_name, AVG(e.salary) AS avg_salary
FROM emp_table e
JOIN department d ON e.dept_id = d.dept_id
GROUP BY d.dept_name;
```

OUTPUT:

```
SQL> CREATE MATERIALIZED VIEW emp_mv
     BUILD IMMEDIATE
     REFRESH COMPLETE
     AS...
Show more...



Materialized view EMP_MV created.

Elapsed: 00:00:00.119
```

REFRESH MATERIALIZED VIEW:

```
EXEC DBMS_MVIEW.REFRESH('EMP_MV');
```

OUTPUT:

```
SQL> EXEC DBMS_MVIEW.REFRESH('EMP_MV')



PL/SQL procedure successfully completed.

Elapsed: 00:00:00.142
```

PERFORMANCE ANALYSIS:

QUERY ON SIMPLE VIEW:

```
SELECT * FROM emp_view;
```

OUTPUT:

	EMP_ID	EMP_NAME	SALARY
1	101	John	3000
2	102	Alice	5000
3	103	Bob	4000
4	104	Emma	3500

QUERY ON COMPLEX VIEW:

```
SELECT * FROM emp_dept_view;
```

OUTPUT:

	DEPT_NAME	AVG_SALARY
1	HR	3250
2	IT	4500

QUERY ON MATERIALIZED VIEW:

```
SELECT * FROM emp_mv;
```

OUTPUT:

	DEPT_NAME	AVG_SALARY
1	HR	3250
2	IT	4500

LEARNING OUTCOMES:

11. Understood the concept of simple, complex, and materialized views
2. Learned how to create and use materialized views in Oracle
3. Compared performance between different types of views
4. Learned how materialized views improve query performance
5. Applied concepts useful in real-world applications

CONCLUSION:

This experiment demonstrated the differences between simple views, complex views, and materialized views. Materialized views improve performance by storing precomputed results, making them highly useful for data-intensive applications.